🎯 MNIST-CUDA

Open in Colab

A custom deep learning framework built from scratch using CUDA in C++. This project implements essential neural network components, including:

- ✅ Linear (fully connected) layers
- ✅ Convolutional layers
- ✅ ReLU activation function
- ✅ Mean Squared Error (MSE) loss
- ✅ Softmax layer
- ✅ Max pooling layer
- ✅ Quantised linear layer (with custom 10-bit float weights)
- ✅ Cross Entropy Loss function

The framework is designed to efficiently train and evaluate models on the MNIST dataset using GPU acceleration for fast computations.

Note: This project is a work in progress and may receive updates in future releases.

🚀 Key Features

- **GPU Acceleration**: Leverages CUDA for parallelized computations, ensuring efficient training and evaluation.
- **Modular Design**: Features a flexible architecture with easily extendable modules for various neural network components.
- **MNIST Dataset Support**: Specifically tailored for training and testing on the MNIST dataset.

📚 Installation

Prerequisites

- **CUDA Toolkit**: Ensure that the CUDA Toolkit is installed and configured correctly for your GPU.
- **C++ Compiler**: A C++17 compatible compiler is required.

Clone the Repository

```
git clone https://github.com/AnimateBow4809/mnist-cuda.git
cd mnist-cuda
```

Building the Project

To build the project, we use `nvcc`, the NVIDIA CUDA Compiler. Ensure that your source files are organized within the `src/` directory and header files within the `include/` directory.

Compilation Command:

```
nvcc -arch=sm_75 -o mnist_cuda \
    src/MNISTTest.cpp \
    src/kernel.cu \
    src/ConvLayer2D.cu \
    src/LinearLayer.cu \
    src/ReluLayer.cu \
    src/DatasetLoader.cu \
    src/MSELoss.cu \
    src/NNModel.cu \
    src/Utils.cu \
    src/softmaxLayer.cu \
    src/MaxPoolLayer.cu \
    src/CrossEntropyLoss.cu \
    src/LinearLayerQuantised.cu \
    src/Trainer.cu \
    -I/usr/local/cuda/include \
    -I./include \
    -L/usr/local/cuda/lib64 \
    -L/usr/lib/x86_64-linux-gnu \
    -lcublas -lcudnn
```

Explanation:

- `-arch=sm_75`: Targets the NVIDIA Volta architecture (sm_75). Adjust this based on your GPU's compute capability.
- `-o mnist_cuda`: Specifies the output executable name.
- List of `src/*.cu` and `src/*.cpp` files: Includes all necessary source files for the project.
- `-I/usr/local/cuda/include`: Adds the CUDA include directory to the search path.
- `-I./include`: Adds the project's include directory to the search path.
- `-L/usr/local/cuda/lib64`: Adds the CUDA library directory to the linker search path.
- `-L/usr/lib/x86_64-linux-gnu`: Includes additional system libraries.
- `-lcublas -lcudnn`: Links against the cuBLAS and cuDNN libraries for optimized linear algebra and deep learning operations.

Note: Ensure that the paths provided (e.g., `/usr/local/cuda/include`) align with your system's CUDA installation directories.

🛠️ Project Structure

```
mnist-cuda/
├── src/
│   ├── ConvLayer2D.cu
```

```
│   ├── CrossEntropyLoss.cu
│   ├── DatasetLoader.cu
│   ├── LinearLayer.cu
│   ├── LinearLayerQuantised.cu
│   ├── MSELoss.cu
│   ├── MaxPoolLayer.cu
│   ├── MNISTTest.cpp
│   └── ReluLayer.cu
├── include/
│   ├── ConvLayer2D.cuh
│   ├── DatasetLoader.cuh
│   ├── Float10.cuh
│   ├── LinearLayer.cuh
│   ├── LinearLayerQuantised.cuh
│   ├── LossFunction.cuh
│   ├── MaxPoolLayer.cuh
│   ├── NNLayer.cuh
│   └── NNModel.cuh
├── archive/
│   └── (archived files)
├── .gitignore
└── README.md
```

Directory Descriptions:

- `src/` : Contains all source files (`.cu` and `.cpp`).
- `include/` : Contains all header files (`.cuh` and `.h`).
- `archive/` : For storing any archived or deprecated files.
- `.gitignore` : Specifies files and directories to be ignored by Git.
- `README.md` : This file.

🏭 Model Architecture

The implemented model consists of the following layers:

1. **Convolutional Layer** (Batch x 1 x 28 x 28 → 32 filters, 3x3 kernel, stride=2, padding=1)
2. **ReLU Activation**
3. **Convolutional Layer** (Batch x 32 x 14 x 14 → 64 filters, 3x3 kernel, stride=2, padding=1)
4. **ReLU Activation**
5. **Convolutional Layer** (Batch x 64 x 7 x 7 → 128 filters, 3x3 kernel, stride=2, padding=1)
6. **ReLU Activation**
7. **Convolutional Layer** (Batch x 128 x 4 x 4 → 32 filters, 3x3 kernel, stride=2, padding=1)
8. **ReLU Activation**
9. **Quantised Linear Layer** (128 → 64)
10. **ReLU Activation**
11. **Quantised Linear Layer** (64 → 32)
12. **ReLU Activation**
13. **Quantised Linear Layer** (32 → 10)
14. **Softmax Layer**

This architecture efficiently processes the MNIST dataset while leveraging CUDA for performance improvements.

🤖 Model Performance Comparison

The model consists of **140,234 weights**. We compared two versions of the same architecture:

1. **Standard FP32 Linear Layers**:
   - Training loss decreased from **2.3050 to 0.2478** over 20 epochs.
   - Final batch accuracy: **93.82%**.
   - Weight range: **-0.6499 to 0.6764**.
2. **Quantised Linear Layers (Float10 Weights)**:
   - Training loss decreased from **2.2738 to 0.2489** over 40 epochs.
   - Final batch accuracy: **93.44%**.
   - Weight range: **-0.6165 to 0.6799**.

This shows that quantised weights perform comparably to standard FP32 weights while using reduced precision, which can benefit memory and computational efficiency.

🧪 Usage

After building the project, you can run the `mnist_cuda` executable to train and evaluate models on the MNIST dataset. Ensure that your system has the necessary permissions to access GPU resources.

🧑‍💻 Contributing

Contributions are welcome! If you'd like to contribute:

1. Fork the repository.
2. Create a new branch (`git checkout -b feature-name`).
3. Commit your changes (`git commit -am 'Add new feature'`).
4. Push to the branch (`git push origin feature-name`).
5. Create a new Pull Request.

Please ensure that your code adheres to the project's coding standards and passes all tests.

📝 License

🙏 Acknowledgments

- **NVIDIA CUDA Toolkit**: For providing the necessary tools and libraries for GPU programming.
- **MNIST Dataset**: For serving as a standard benchmark for machine learning algorithms.